

Aws2.py

```
import boto3
```

```
import json
```

```
def lambda_handler(event, context):
```

```
    """
```

```
    This function stores a document or PDF file in an S3 bucket.
```

```
    Args:
```

```
        event: The event dict that AWS Lambda provides.
```

```
        context: The context in which the function is called.
```

```
    Returns:
```

```
        The API Gateway Lambda Proxy Output Format containing the result.
```

```
    """
```

```
    try:
```

```
        bucket_name = event['bucket_name']
```

```
        key = event['key']
```

```
        file_body = event['file_body']
```

```
        s3 = boto3.client('s3')
```

```
        s3.put_object(Bucket=bucket_name, Key=key, Body=file_body)
```

```
        return {
```

```
            'statusCode': 200,
```

```
            'body': json.dumps({'message': 'File uploaded successfully'})
```

```
        }
```

```
    except Exception as e:
```

```
        return {
```

```
            'statusCode': 500,
```

```
            'body': json.dumps({'error': str(e)})
```

```
        }
```

aws.py

```
import json
```

```
def lambda_handler(event, context):
```

```
    """
```

```
    This function adds two numbers and returns the result.
```

```
    Args:
```

```
        event: The event dict that AWS Lambda provides.
```

```
        context: The context in which the function is called.
```

Returns:

The API Gateway Lambda Proxy Output Format containing the sum.

"""

try:

```
    num1 = event['num1']
    num2 = event['num2']
    result = num1 + num2
    return {
        'statusCode': 200,
        'body': json.dumps({'sum': result})
    }
```

except KeyError as e:

```
    return {
        'statusCode': 400,
        'body': json.dumps({'error': f'Missing parameter: {e}'})
    }
```

Overall.html

```
<ul>
    {% for user in users %}
        <li><a href="{% url 'chat_room' user.id %}">{{ user.username }}</a></li>
    {% endfor %}
</ul>
```

```
<div>
    <ul id="messages">
        {% for message in messages %}
            <li>
                {{ message.sender.username }}: {{ message.content }}
                <span class="timestamp">{{ message.timestamp }}</span>
            </li>
        {% endfor %}
    </div>
```

B1.py

```
from channels.generic.websocket import AsyncWebsocketConsumer
```

```
class ChatConsumer(AsyncWebsocketConsumer):
```

```
    async def connect(self):
```

```
        self.room_name = self.scope['url_route']['kwargs']['room_name']
```

```
        self.room_group_name = 'chat_%s' % self.room_name
```

```
        await self.channel_layer.group_add(
```

```

        self.room_group_name,
        self.channel_name
    )

    await self.accept()

    async def disconnect(self, close_code):
        await self.channel_layer.group_discard(
            self.room_group_name,
            self.channel_name
        )

    async def receive(self, text_data=None, bytes_data=None):
        # ... handle incoming messages ...

    async def chat_message(self, event):
        message = event['message']
        await self.send(text_data=json.dumps(message))

```

B.py

```

from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.models import User
from .models import Message
from django.http import JsonResponse
from channels.layers import get_channel_layer
from asgiref.sync import async_to_sync

def signup(request):
    # ... signup logic ...

def login_view(request):
    # ... login logic ...

def logout_view(request):
    # ... logout logic ...

def chat_list(request):
    users = User.objects.exclude(id=request.user.id)
    return render(request, 'chat/chat_list.html', {'users': users})

def chat_room(request, user_id):
    recipient = User.objects.get(id=user_id)
    messages = Message.objects.filter(sender=request.user, recipient=recipient) | \
        Message.objects.filter(sender=recipient, recipient=request.user)
    return render(request, 'chat/chat_room.html', {'recipient': recipient, 'messages': messages})

```

```

def send_message(request):
    if request.method == 'POST':
        recipient_id = request.POST.get('recipient')
        recipient = User.objects.get(id=recipient_id)
        message = Message.objects.create(
            sender=request.user,
            recipient=recipient,
            content=request.POST.get('message')
        )

        # Send message to WebSocket channel
        channel_layer = get_channel_layer()
        async_to_sync(channel_layer.group_send)(
            f'chat_{recipient_id}',
            {
                'type': 'chat_message',
                'message': {
                    'sender': message.sender.username,
                    'content': message.content,
                    'timestamp': message.timestamp.strftime('%b %d, %Y, %I:%M %p')
                }
            }
        )

    return JsonResponse({'status': 'success'})

```

Backend.py

```

from django.db import models
from django.contrib.auth.models import User

class Message(models.Model):
    sender = models.ForeignKey(User, on_delete=models.CASCADE,
        related_name='sent_messages')
    recipient = models.ForeignKey(User, on_delete=models.CASCADE,
        related_name='received_messages')
    content = models.TextField()
    timestamp = models.DateTimeField(auto_now_add=True)

```

Web.js

```

const toggleButton = document.querySelector('.toggle-button');
const leftMenu = document.querySelector('.left-menu');

toggleButton.addEventListener('click', () => {

```

```

    leftMenu.classList.toggle('active');
  });

// Function to shrink the page based on screen width
function shrinkPage() {
    const screenWidth = window.innerWidth;

    if (screenWidth >= 992 && screenWidth <= 1600) {
        document.body.style.transform = 'scale(0.9)';
    } else if (screenWidth >= 700 && screenWidth <= 767) {
        document.body.style.transform = 'scale(0.8)';
    } else if (screenWidth >= 600 && screenWidth < 700) {
        document.body.style.width = '75%';
    } else if (screenWidth <= 600) {
        document.body.style.width = '50%';
    }
}

// Call the shrinkPage function on initial load and on window resize
shrinkPage();
window.addEventListener('resize', shrinkPage);

```

Web.css

```

body {
    margin: 0;
    font-family: sans-serif;
}

.navbar {
    position: fixed;
    top: 0;
    width: 100%;
    background-color: #333;
    color: #fff;
    z-index: 1;
}

.container {
    max-width: 960px;
    margin: 0 auto;
    padding: 20px;
}

.left-menu {
    background-color: #f0f0f0;
    padding: 20px;
}

```

```

    height: 100vh;
    overflow-y: auto;
}

.toggle-button {
    display: none;
}

.main-content {
    flex: 1;
    padding: 20px;
}

.right-panel {
    background-color: #f0f0f0;
    padding: 20px;
    height: 100vh;
    overflow-y: auto;
}

/* Responsive Styles */
@media (max-width: 768px) {
    .left-menu {
        display: none;
    }

    .toggle-button {
        display: block;
    }

    .container {
        flex-direction: column;
    }
}

```

Web.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Responsive Webpage</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

```

```
<header>
  <nav class="navbar">
    <div class="container">
      <ul>
        <li><a href="#">Home</a></li>
        <li><a href="#">About</a></li>
        <li><a href="#">Contact</a></li>
      </ul>
    </div>
  </nav>
</header>

<main>
  <div class="container">
    <aside class="left-menu">
      <button class="toggle-button">Toggle Menu</button>
      <ul>
        <li><a href="#">Menu Item 1</a></li>
        <li><a href="#">Menu Item 2</a></li>
        <li><a href="#">Menu Item 3</a></li>
      </ul>
    </aside>
    <section class="main-content">
      <h1>Main Content Area</h1>
      <p>This is the main content area.</p>
    </section>
    <aside class="right-panel">
      <h2>Right Panel</h2>
      <p>This is the right-side panel.</p>
    </aside>
  </div>
</main>

<footer>
  <p>© 2023 My Website</p>
</footer>

  <script src="script.js"></script>
</body>
</html>
```